Version 1.16
10/07/2026

NMC Bring-Up Guide: Cabling, Programming & Loading Linux (Yocto)

WARNING: Wear an ESD wrist strap whenever you handle the NMC board. Electrostatic discharge can permanently damage the NMC board.

Overview

Going from a bare NMC board to a booting Linux prompt takes eight steps and three connections to your host PC: a programming connection for the FPGA design and firmware, USB-to-serial connections for the two serial consoles, and one Ethernet cable for TFTP.

1. Install Libero SoC, its license, and SoftConsole.
2. Clone the Nextage `Project-NMC` repo and learn which MSS output files go where.
3. Cable the board and open two serial terminals.
4. Program the FPGA design and firmware (Libero, MSS Configurator, SoftConsole).
5. Build the Linux images with Yocto.
6. Flash `payload-yocto.bin` to eMMC from the HSS prompt (YMODEM).
7. Flash `yocto.wic.gz` to eMMC from U-Boot (TFTP).
8. Boot Linux.

Have on hand:

- The NMC board and a 5 V power supply, current-limited to 2 A
- A Windows host with WSL2 / Ubuntu 22.04 LTS (the tested setup for the Linux build)
- Tera Term, a TFTP server such as tftpd64, and GitHub Desktop
- A Microchip account for the Libero license download
- The cables listed in Step 3

Step 1: Install the software

Install Libero SoC (with its node-locked license) and SoftConsole on the host PC; both use default download settings.

Libero SoC Design Suite (includes the MSS Configurator)

1. Download Libero SoC from Microchip's FPGA Software Downloads page. Keep the default settings and pick any install directory.
2. Libero needs a license tied to your machine. Generate it from Microchip's FPGA Software Licenses page and follow the *Libero SoC License Installation Guide* for the **1 year node-locked Silver license**.
3. Node-locked means the license is bound to this one device. Floating licenses are for servers.
4. Once the license is installed and linked, Libero SoC should launch and run.

SoftConsole

1. Download SoftConsole from Microchip's SoftConsole page.
2. Keep the default settings and pick any install directory.

Ubuntu 22.04 under WSL2 (for the Linux build)

1. Open PowerShell and check what is installed: `wsl -l -v`.
2. If Ubuntu 22.04 is not listed, install it: `wsl --install -d Ubuntu-22.04`.
3. Open Ubuntu 22.04 from the Start menu and create a username and password.
4. In the Ubuntu window, check the version: `cat /etc/os-release`. It should say Ubuntu 22.04.
5. Optional: run `wsl --set-default Ubuntu-22.04` in PowerShell so plain `wsl` opens Ubuntu.

Use Ubuntu 22.04 rather than Debian or the docker-desktop distro, which lack some packages or are untested for these builds. Work in the Ubuntu home folder (`cd ~`), not under `/mnt/c`.

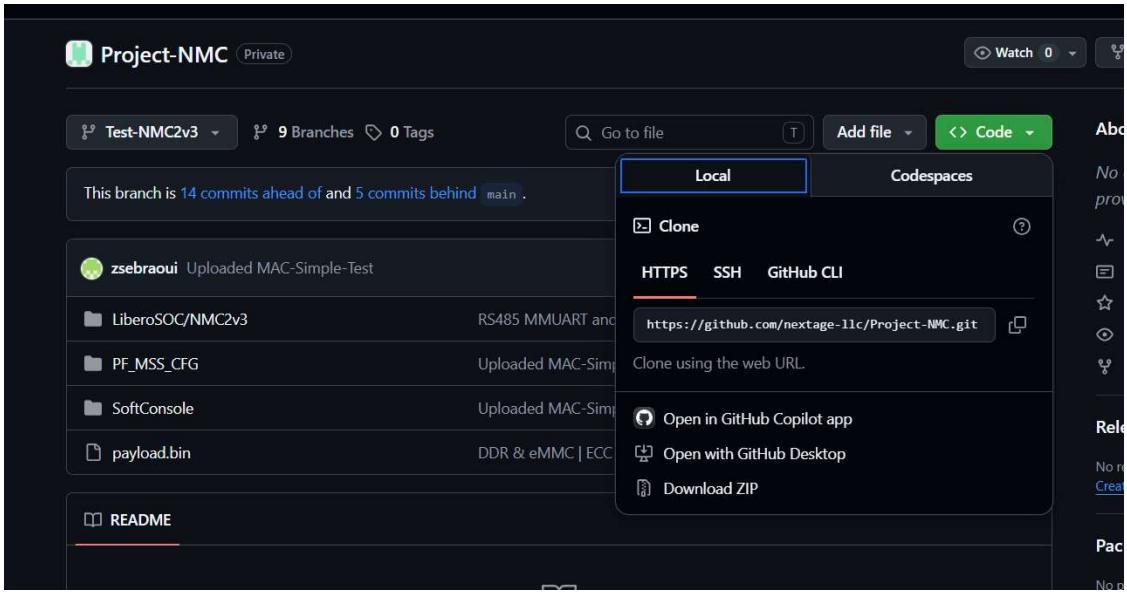
Links

- Libero SoC download (FPGA Software Downloads)
  https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/fpga-software-downloads
- Libero license generation (FPGA Software Licenses)
  https://www.microchipdirect.com/fpga-software-products
- Libero SoC License Installation Guide
  https://ww1.microchip.com/downloads/aemDocuments/documents/FPGA/swdocs/libero/Libero_Installation_Licensing_Setup_User_Guide_2025_1.pdf
- SoftConsole download
  https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/soc-fpga/softconsole

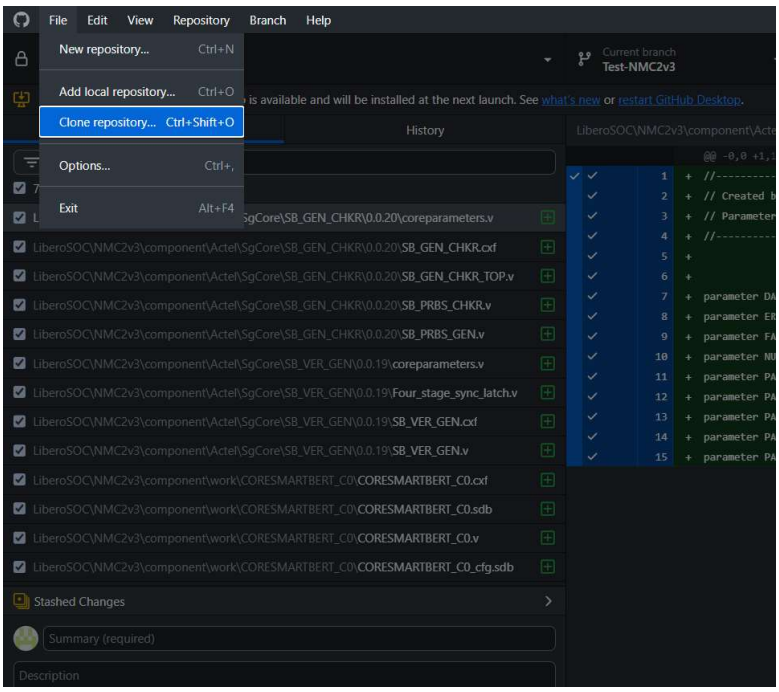
Step 2: Get the code and know your files

Clone one branch of the Nextage `Project-NMC` repo into an empty folder; Install GitHub Desktop.

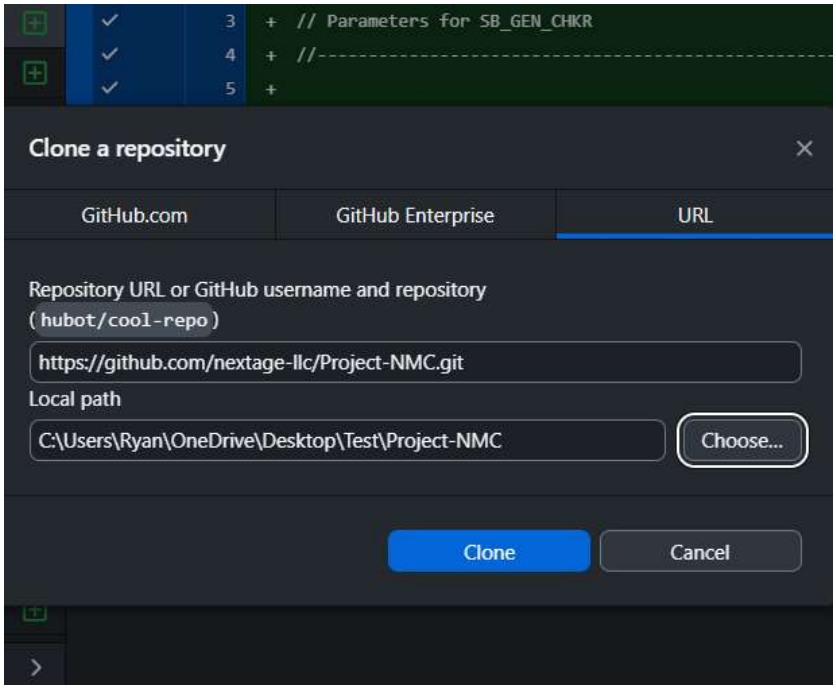
1. Choose **File → Clone repository → URL** and enter `https://github.com/nextage-llc/NMC_OS_BSP.git`.
2. Set the local path to a new, empty folder.
3. Pick the branch you need. `main` is the baseline and the others are test branches (the guide's screenshots use `Test-NMC2v3`). Click **Clone**.



On the repo page, the Code menu shows the clone URL and an "Open with GitHub Desktop" option. The branch selector at top left picks the branch (here Test-NMC2v3).



In GitHub Desktop, open File → Clone repository (Ctrl+Shift+O).



On the URL tab, paste the repo URL, set Local path to your empty folder, and click Clone.

Each branch contains `LiberoSOC/NMC2v3`, `PF_MSS_CFG`, `SoftConsole`, and a `payload.bin`.

The MSS Configurator produces files that both Libero and SoftConsole consume. The `.hex` comes from an IDE such as SoftConsole instead.

File	What it is	Where it goes
`.cfg`	GUI settings file for the RISC-V MSS. Not used in firmware generation.	Open it in the MSS Configurator to edit MSS settings. Press **Generate** to produce the `.cxz` and `.xml`.
`.cxz`	MSS Configurator output	Libero SoC design flow: **Create Design → Import MSS**
`.xml`	MSS Configurator output; carries the hardware configuration to the RISC-V processors	SoftConsole: drag it into the project and delete the default `.xml`, so only the new one remains
`.hex`	Built in an IDE (SoftConsole); used for bootloading and memory initialization	Libero SoC: **Program Design → Configure Design Initialization Data and Memories → eNVM**

The separate Linux BSP lives in its own repo, `mpfs-nmc-linux-bsp`, covered in Step 5.

Step 3: Cable the board and open the serial terminals

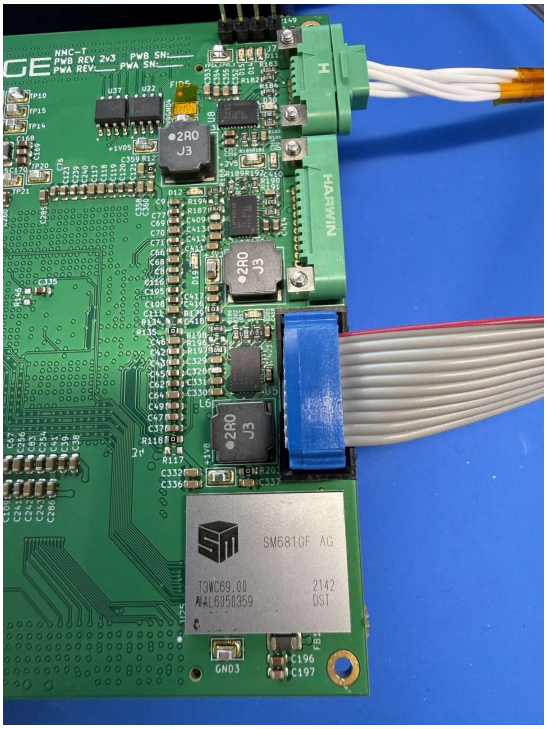
Connect the board to the host PC with USB-to-serial cabling into the UART0 and UART1 headers for the consoles and one Ethernet cable for TFTP, plus a Microchip FlashPro programmer and a 5 V power feed (photos below).

Connection	From	To	Used for	Source detail
Power	5 V supply	Board (J7)	Powering and power-cycling the board	Photo below
Programming link	Host PC (Libero), then USB to FlashPro5 or FlashPro6	Board (J2)	Programming the FPGA design and firmware (Step 4)	Photos below
USB	Host PC (USB)	Board (UART0 and UART1 headers)	Two serial ports: UART0 (HSS bootloader) and UART1 (U-Boot and Linux console)	Photo below
Ethernet	Host PC	Board (J19)	TFTP transfer of `yocto.wic.gz` (Step 7)	Photo below. Direct to the host.

Connect the programmer. Plug the FlashPro5 or FlashPro6 into a USB port on the host PC, then plug its ribbon cable into the J2 connector on the NMC's edge.

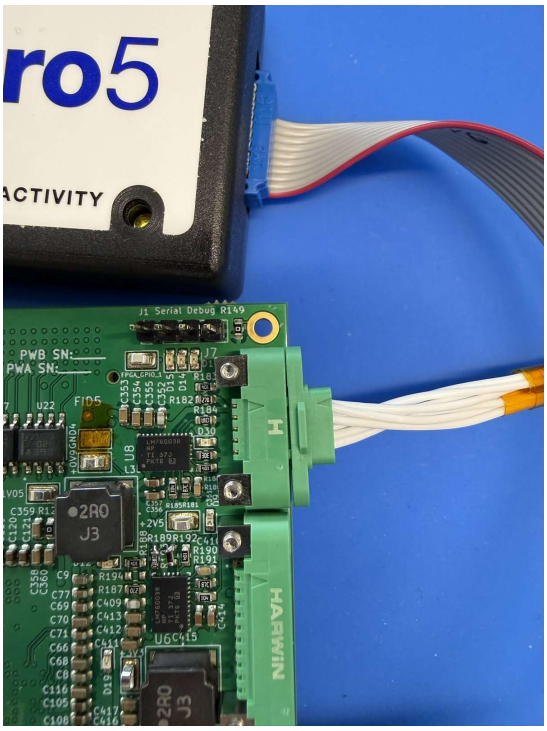


The FlashPro5 (a FlashPro6 works the same way). The USB cable to the host PC leaves on the left, and the ribbon cable to the NMC leaves on the right.



The ribbon cable seats in the J2 connector on the board edge. Match the orientation in the photo, with the cable's red-stripe edge on top.

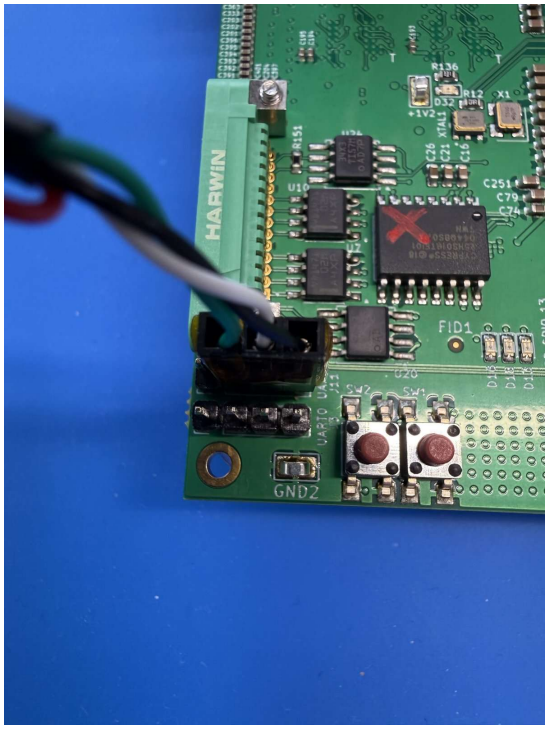
Connect power. Feed 5V into connector J7 on the board edge. Set the supply's current limit to 2 A. There are no jumpers or boot-mode settings to set and no power-up order: turn on the supply and the board sequences its own internal rails.



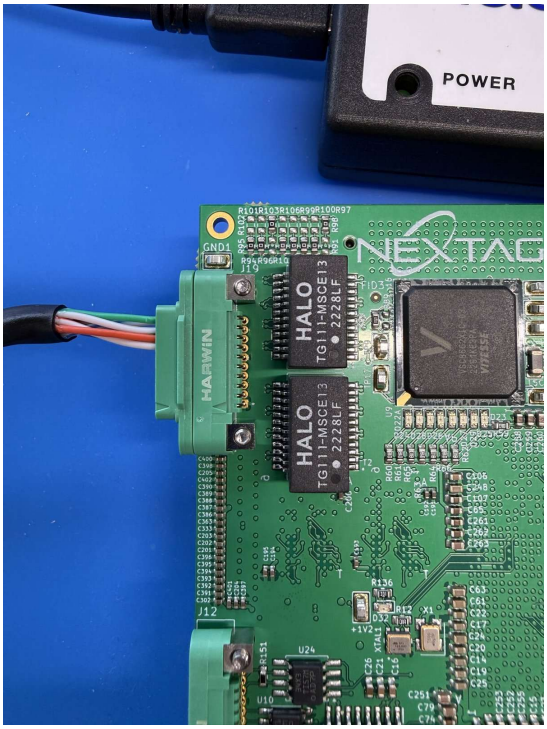
Power goes into J7, shown here with the white wire bundle plugged in.

Connect the serial consoles. Connect the USB-to-serial cable to the board's UART headers, UART0 for the HSS bootloader and UART1 for U-Boot and Linux, then plug the USB end into the host PC.

Connect Ethernet. Plug the Ethernet cable into the board's J19 connector, and run the other end to the host PC.



Serial consoles. The UART headers sit together near the board's lower-left corner: UART0 (J3) and UART1 (J11). The photo shows a USB-to-serial connector seated on UART1.

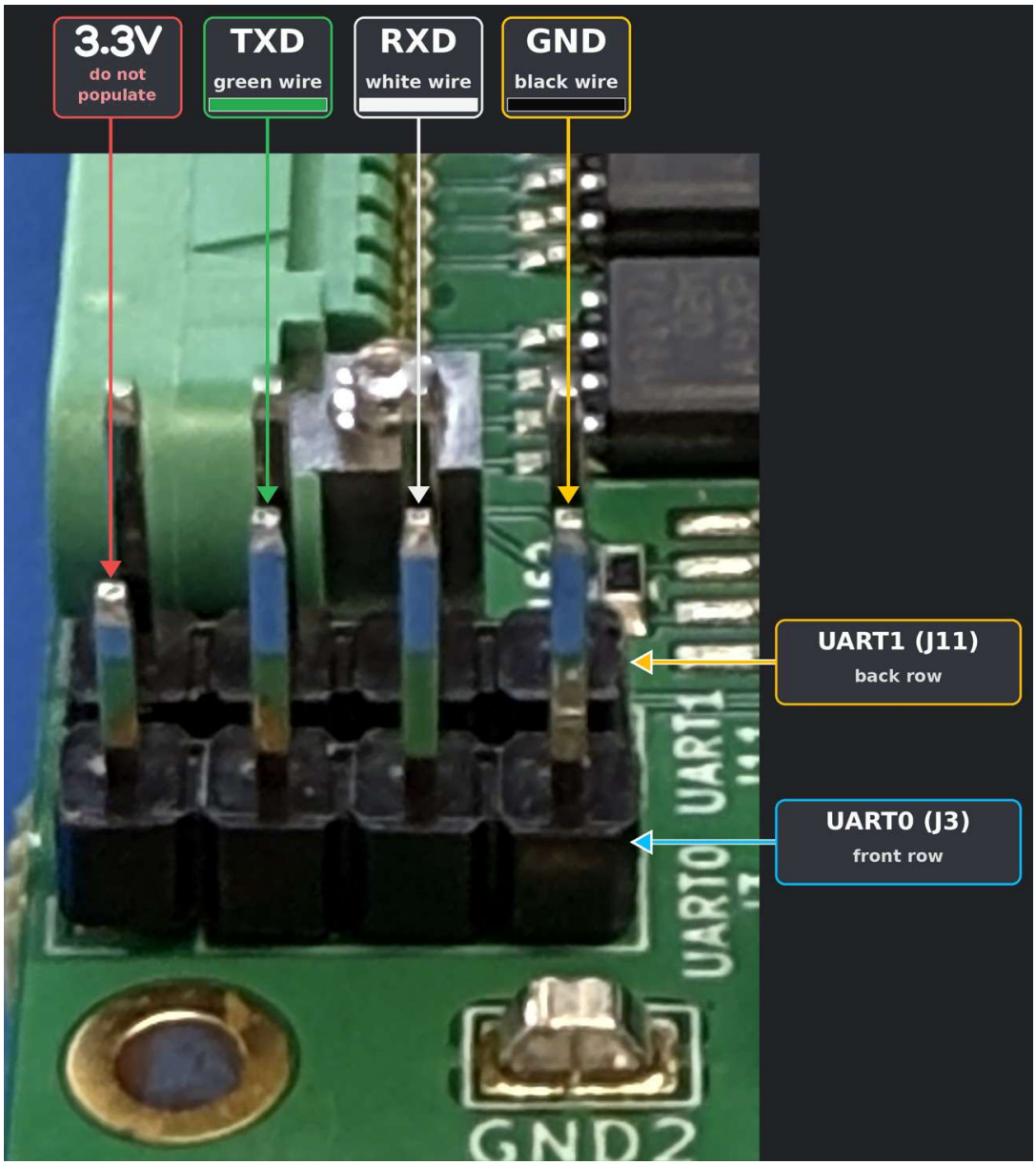


Ethernet. The cable goes into connector J19 at the board edge, next to the two HALO magnetics modules.

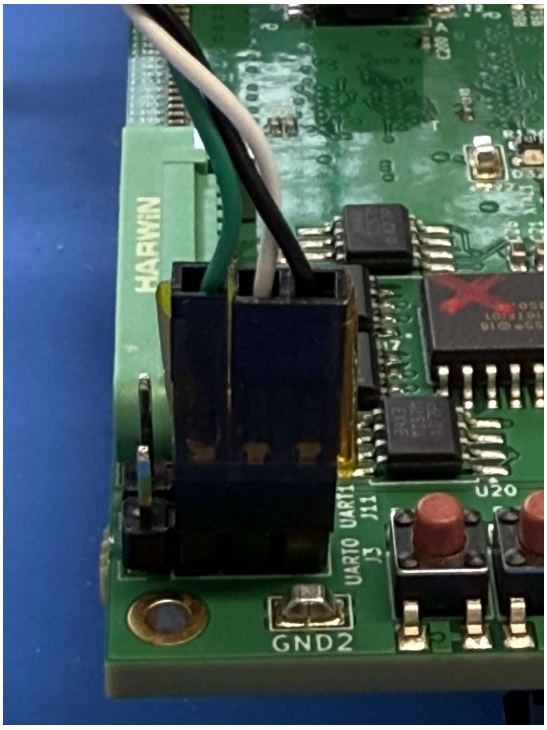
Serial cable wiring. The cable is a 3.3 V TTL USB-to-UART cable (PL2303TA, USB A male, 1x4 female 2.54 mm socket). It carries one UART, so use one cable per UART header. The board header takes only three of the four wires, so leave the red 5 V wire out of the socket.

Header position (left to right, as photographed)	Board signal	Cable wire	Note
1	3.3V	Red (VCC)	Do not populate
2	TXD	Green	Cable RXD
3	RXD	White	Cable TXD
4	GND	Black	Cable GND

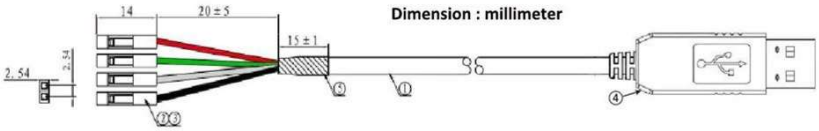
Signals are named from the board's side, so the board's TXD meets the cable's RXD.



The two UART headers, labeled. Both use the same pin order, left to right as photographed with the board edge at the bottom: 3.3V (leave unpopulated), TXD, RXD, GND. The back row is UART1 (J11) and the front row is UART0 (J3).



The cable's socket seated on a UART header. The green, white, and black wires sit on pins 2 to 4, and the leftmost pin (3.3V) is left open.



① PVC Jacket
② 2.54mm pitch socket
③ 2.54mm 1*4p terminal ,female
④ USB A male, PL2303TA TTL 3.3V PCBA inside
⑤ Heat shrink tube, black, OD=5.0

1*4P Female Socket	Name	Colour	Description
Pin 1	TXD	White	Transmit Asynchronous Data
Pin 2	RXD	Green	Receive Asynchronous Data
Pin 3	GND	Black	Device ground supply
Pin 4	VCC	Red	+5V



The USB-to-UART cable and its own pin table. Its socket pins are numbered differently from the board header, so use the table above for placement.

Open two Tera Term windows, one per COM port that the USB connection creates. It is also acceptable to use one Tera Term window and just swap between the 2 UARTs. Tera Term will automatically pick up the 2nd UART comms without any intervention.

Use these settings on both:

- Baud: 115200
- Data: 8 bit
- Parity: none
- Stop: 1 bit
- Flow control: none

Window	COM port	Shows
Tera Term Window 1 (UART0)	Port showing the HSS boot log	UART0: HSS bootloader, hardware initialization and boot logs
Tera Term Window 2 (UART1)	The other port	UART1: OpenSBI banner, U-Boot prompt, then the Linux console

Which port is which. Windows assigns the COM numbers itself, and they differ between PCs and USB hubs, so identify the ports by what they print once the board is programmed (Step 4). Open both, then reset the board. The window that prints the HSS boot messages is UART0 (Window 1). The other is UART1 (Window 2), which prints DDR training and the OpenSBI banner, then the U-Boot prompt.

Step 4: Program the FPGA design and firmware

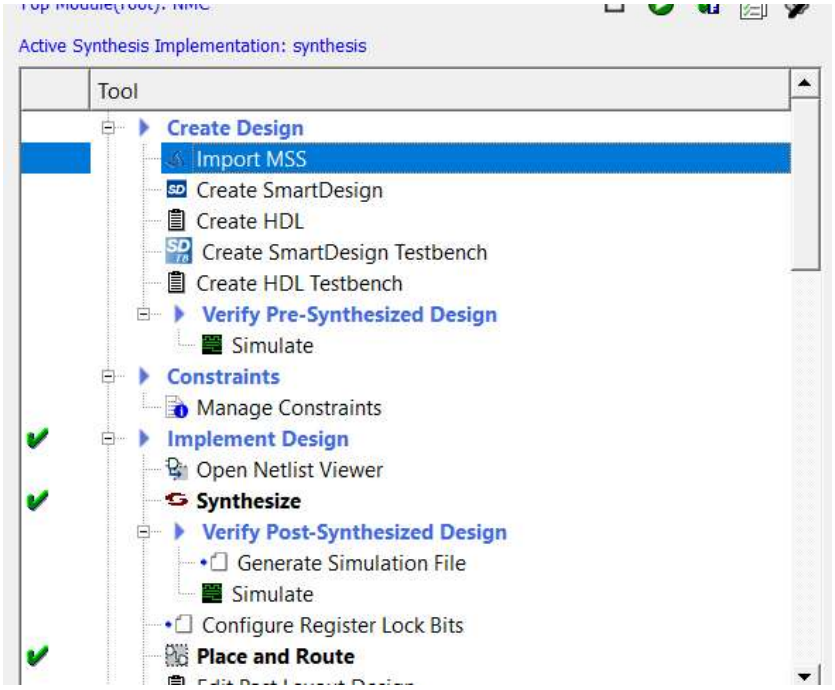
Edit the MSS configuration, build the Libero design, write application code in SoftConsole, and program the result to the board.

A. Configure the MSS

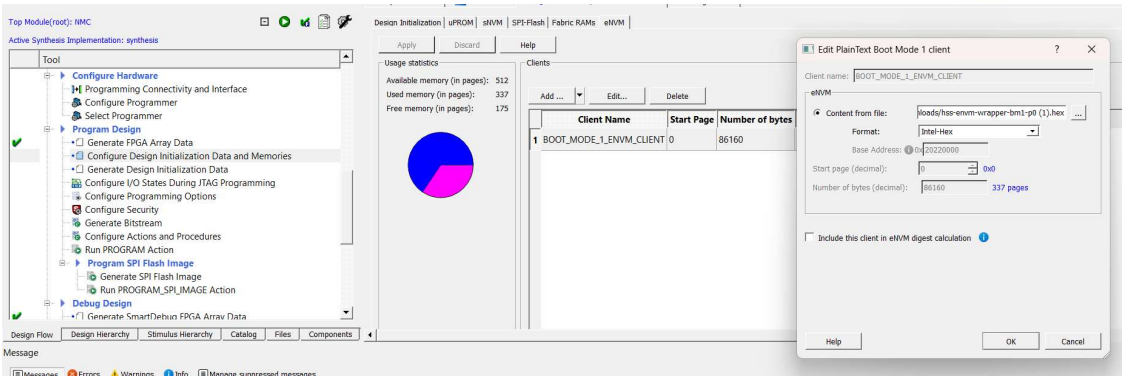
1. Open the `.cfg` file from the `PF_MSS_CFG` folder in the MSS Configurator and edit the RISC-V MSS settings.
2. Press **Generate** to write the `.cxz` and `.xml` output files.

B. Build the design in Libero SoC

1. In the design flow, run **Create Design → Import MSS** and select the new `.cxz`.
2. Work down the flow as shown in the guide's screenshots: **Create SmartDesign**, **Manage Constraints**, **Synthesize**, **Place and Route**.
3. Under **Program Design → Configure Design Initialization Data and Memories → eNVM**, add or edit a client (the guide's example is `BOOT_MODE_1_ENVM_CLIENT`). Choose **Content from file**, format **Intel-Hex**, and select your `.hex`.



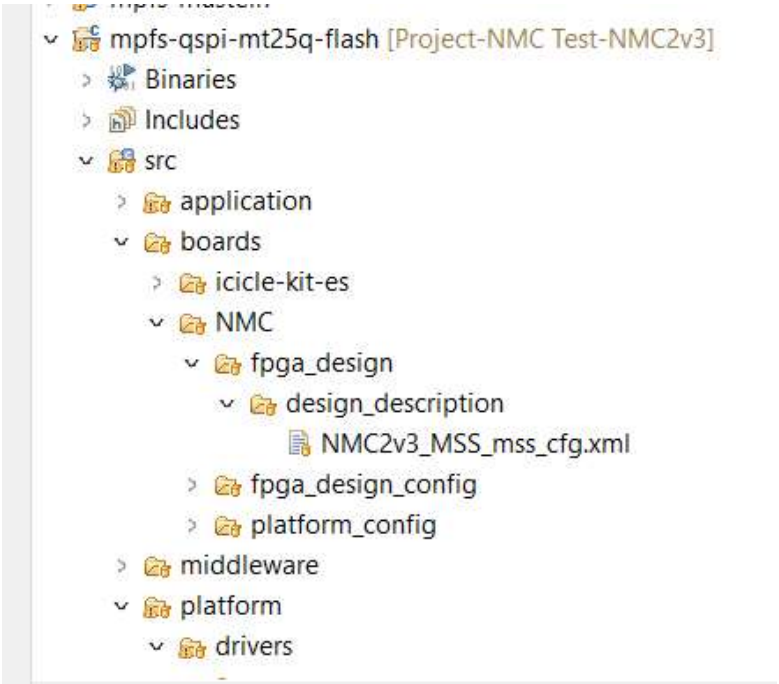
Libero design flow: Create Design → Import MSS is where the `.cxz` goes.



The `.hex` import location: Program Design → Configure Design Initialization Data and Memories → eNVM, then Add a client and select the Intel-Hex file.

C. Write application code in SoftConsole

1. Open the SoftConsole project from the `SoftConsole` folder. The guide's example is `mpfs-qspi-mt25q-flash`.
2. Drag the new `.xml` into the project's `fpga_design/design_description` folder and delete the default `.xml`, so only one remains.
3. Put application code under `src/application`. Hart0 (`e51.c`) mainly handles system startup, so use hart1 to hart4 (`u54_1.c` to `u54_4.c`) for your code.
4. Build to produce the `.hex`, then import it into Libero as in B3.



SoftConsole: the MSS `.xml` sits under `src/boards/NMC/fpga_design/design_description`. Replace the default one with yours.



Application code lives in `src/application`. Hart0 (`e51.c`) handles startup, so use hart1 to hart4.

D. Program the board

1. Connect the FlashPro to the host PC (USB) and to the NMC's ribbon-cable connector, as pictured in Step 3.
2. In Libero, under **Program Design**, run **Generate FPGA Array Data** and **Generate Bitstream**.
3. Run **Run PROGRAM Action** to write the design and eNVM contents to the board.

Step 5: Build the Linux images with Yocto

This step builds the Linux images with Yocto/OpenEmbedded. It produces a `payload.bin` (HSS-wrapped U-Boot) and a full-disk image (`.wic`) that you flash in Steps 6 to 8. Microchip's current Yocto BSP is `meta-mchp` (Yocto release scarthgap). The older `meta-polarfire-soc-yocto-bsp` repo was archived in April 2026, so do not use it. The NMC uses the same MPFS250T chip as the Icicle Kit, so the `mpfs-icicle-kit` machine is the starting point.

Plan on roughly 100 GB of free disk and 16 GB of RAM or more; the first build takes several hours. Build inside the WSL2 Linux filesystem (for example `~/nmc-yocto`), not under `/mnt/c/...`.

A. Install host dependencies (WSL2 / Ubuntu 22.04 LTS)

```
sudo apt update
sudo apt-get install gawk wget git git-lfs diffstat unzip texinfo gcc \
  build-essential chrpath socat cpio python3 python3-pip \
  python3-pexpect xz-utils debianutils iputils-ping python3-git \
  python3-jinja2 python3-subunit zstd lz4 file locales \
  libacl1 libgl1 repo bmap-tools
sudo locale-gen en_US.UTF-8
```

B. Fetch the Microchip Yocto BSP

Set your git identity once, since `repo init` asks for it:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git lfs install

mkdir -p ~/nmc-yocto && cd ~/nmc-yocto
repo init -u https://github.com/linux4microchip/meta-mchp-manifest.git \
  -b refs/tags/linux4microchip-2026.04 -m polarfire-soc/default.xml
repo sync
```

`linux4microchip-2026.04` was the latest release tag when this was written. Check the `meta-mchp` Releases page on GitHub for a newer one.

C. Initialize the build environment

```
cd ~/nmc-yocto
export TEMPLATECONF=${TEMPLATECONF:-../meta-mchp/meta-mchp-polarfire-soc/meta-mchp-
polarfire-soc-bsp/conf/templates/default}
source openembedded-core/oe-init-build-env
```

`TEMPLATECONF` is only needed the first time. In every new shell, run `source openembedded-core/oe-init-build-env` again from `~/nmc-yocto` before using `bitbake`.

D. Build the baseline Icicle Kit image.

```
MACHINE=mpfs-icicle-kit bitbake mchp-base-image
```

Outputs land in `build/tmp-glibc/deploy/images/mpfs-icicle-kit/`. The `.wic` file there is the disk image. Also look in that directory for the HSS `payload.bin`. If it is missing, find the HSS payload recipe with `bitbake-layers show-recipes "*hss*"` and build it. Log in as `root` with no password.

E. Add the NMC device trees in your own layer. Yocto re-fetches and re-patches its sources, so do not copy files into the build tree. Put the NMC changes in a layer called `meta-nmc` that overwrites the Icicle

Kit device trees. Keep building with `MACHINE=mpfs-icicle-kit`. Do not create a custom machine, because Microchip's recipes look up files by machine name.

E1. Create the layer and copy in your device trees. Replace `<path-to>` with the Windows folder that contains `mpfs-nmc-linux-bsp` (the same folder as in part G below).

```
cd ~/nmc-yocto
source openembedded-core/oe-init-build-env
cd ~/nmc-yocto
bitbake-layers create-layer meta-nmc
bitbake-layers add-layer ~/nmc-yocto/meta-nmc
mkdir -p meta-nmc/recipes-kernel/linux/files meta-nmc/recipes-bsp/u-boot/files
BSP=/mnt/c/<path-to>/mpfs-nmc-linux-bsp
cp $BSP/linux/mpfs-nmc-linux.dts meta-nmc/recipes-kernel/linux/files/
cp $BSP/uboot/mpfs-nmc-uboot.dts meta-nmc/recipes-bsp/u-boot/files/
```

If you open a new shell, run the `BSP=` line again before using `$BSP`. Check the layer and files:

```
bitbake-layers show-layers | grep nmc
ls -l meta-nmc/recipes-kernel/linux/files meta-nmc/recipes-bsp/u-boot/files
```

E2. Check the inputs match the Yocto build.

```
head -3 $BSP/linux/.config
ls ~/nmc-yocto/build/tmp-glibc/work/mpfs_icicle_kit-mchp-linux/u-boot-
mchp/*/git/dts/upstream/src/riscv/microchip/
```

The first command should name the same kernel version that Yocto builds (6.18.17 at the time of writing). A different version may need changes to the device tree. The second should list `mpfs-icicle-kit.dts`, which is the U-Boot file the overlay overwrites. It exists after the baseline build in part D.

E3. Write the two bbappend files. The recipes are `linux-mchp` and `u-boot-mchp`. The `%` in the filenames matches any version, so the files keep working when Microchip updates them.

The two files below use these variables:

- `${THISDIR}`: the folder that holds the `.bbappend` file itself, for example `meta-nmc/recipes-kernel/linux/`.
- `FILESEXTRAPATHS`: the list of folders BitBake searches for the `file://` entries in `SRC_URI`. Prepending `${THISDIR}/files` makes BitBake look in the `files` folder next to the `.bbappend`, where you copied the DTS.
- `${WORKDIR}`: the recipe's working folder in the build tree (`build/tmp-glibc/work/.../<recipe>/`). The fetcher copies each `file://` item into it in this Yocto release.
- `${S}`: the recipe's source tree, which is the kernel or U-Boot checkout. `do_configure:prepend` copies the DTS from `${WORKDIR}` over `mpfs-icicle-kit.dts` here.

```
cat > meta-nmc/recipes-kernel/linux/linux-mchp_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI:append = " file://mpfs-nmc-linux.dts"
do_configure:prepend() {
    cp ${WORKDIR}/mpfs-nmc-linux.dts ${S}/arch/riscv/boot/dts/microchip/mpfs-icicle-kit.dts
    cp ${WORKDIR}/mpfs-nmc-linux.dts ${S}/arch/riscv/boot/dts/microchip/mpfs-icicle-kit-
prod.dts
}
EOF
```

The NMC U-Boot device tree releases both Ethernet PHYs from reset with `gpio-hog` nodes on `gpio2`. Microchip's stock U-Boot configuration does not include the GPIO driver or hog support, so without the fragment below U-Boot ignores those nodes, the PHYs stay in reset, and TFTP fails with `PHY not found`. Create the fragment first, then the U-Boot bbappend:

```
cat > meta-nmc/recipes-bsp/u-boot/files/nmc-uboot.cfg << 'EOF'
CONFIG_DM_GPIO=y
CONFIG_CMD_GPIO=y
CONFIG_GPIO_HOG=y
CONFIG_MPFS_GPIO=y
EOF

cat > meta-nmc/recipes-bsp/u-boot/u-boot-mchp_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI:append = " file://mpfs-nmc-uboot.dts file://nmc-uboot.cfg"
do_configure:prepend() {
    cp ${WORKDIR}/mpfs-nmc-uboot.dts ${S}/dts/upstream/src/riscv/microchip/mpfs-icicle-
kit.dts
}
EOF
```

E4. Check that bitbake picked them up.

```
bitbake -e virtual/kernel | grep ^SRC_URI=
bitbake -e virtual/bootloader | grep ^SRC_URI=
```

The kernel line should end with `file://mpfs-nmc-linux.dts`, and the U-Boot line should end with `file://mpfs-nmc-uboot.dts file://nmc-uboot.cfg`. If either is missing, the layer is not added or a filename is wrong.

E5. Check that the U-Boot options were applied. Force the configure step again and look for the four options (a `tainted from a forced run` warning is expected):

```
MACHINE=mpfs-icicle-kit bitbake -f -c configure u-boot-mchp
YCFG=$(find ~/nmc-yocto/build/tmp-glibc/work/mpfs_icicle_kit-mchp-linux/u-boot-mchp -name
.config | head -1)
grep -E 'CONFIG_(DM_GPIO|CMD_GPIO|GPIO_HOG|MPFS_GPIO)[ =]' $YCFG
```

All four should show `=y`. If any show `# ... is not set`, the fragment was not merged.

Your NMC U-Boot options are applied by this fragment, and your device trees are replaced by the overlay. The NMC kernel configuration options have not been added yet. In Yocto they are added as small `.cfg` fragments on top of the default configuration, not as complete `.config` files. Microchip also builds device tree overlays for this machine that assume Icicle Kit nodes, so check the boot log for overlay errors.

F. Rebuild. This is much faster than the first build, because only the kernel, U-Boot, the HSS payload and the image are rebuilt.

```
cd ~/nmc-yocto/build
MACHINE=mpfs-icicle-kit bitbake mchp-base-image
```

When it finishes, look for `There were 0 ERROR messages`. Then check that the new images are newer than the baseline ones. The files at the top of the list should have today's date:

```
ls -lt ~/nmc-yocto/build/tmp-glibc/deploy/images/mpfs-icicle-kit | head
```

To confirm the NMC device trees were applied, compare them with the copies in the build trees. No output means they are identical.

```
cd ~/nmc-yocto
diff -q meta-nmc/recipes-kernel/linux/files/mpfs-nmc-linux.dts build/tmp-
glibc/work/mpfs_icicle_kit-mchp-linux/linux-mchp/*/git/arch/riscv/boot/dts/microchip/mpfs-
icicle-kit.dts
diff -q meta-nmc/recipes-bsp/u-boot/files/mpfs-nmc-uboot.dts build/tmp-
glibc/work/mpfs_icicle_kit-mchp-linux/u-boot-
mchp/*/git/dts/upstream/src/riscv/microchip/mpfs-icicle-kit.dts
```

Troubleshooting: Bad address errors in do_package. If a rebuild stops with `do_package` errors that include `got *at() syscall for unknown directory` and `tar: ... Cannot open: Bad address`, the host `tar` is failing under Yocto's fake-root tool. On our machine this started after Ubuntu 22.04 updated `tar` to `1.34+dfsg-1ubuntu0.1.22.04.6`; an earlier build had packaged without errors. Rebuilding single recipes or pseudo did not help. Use the `tar` that Yocto supplies in its prebuilt host tools instead:

```
cd ~/nmc-yocto
./openembedded-core/scripts/install-buildtools
source ~/nmc-yocto/openembedded-core/buildtools/environment-setup-x86_64-pokysdk-linux
source openembedded-core/oe-init-build-env
rm -rf ~/nmc-yocto/build/tmp-glibc/hosttools
tar --version | head -1
```

The last command should report `tar` 1.35. Deleting `hosttools` is needed because BitBake keeps links to the host programs it found on an earlier run, and they would still point to the system `tar`. After rerunning the build, `ls -l ~/nmc-yocto/build/tmp-glibc/hosttools/tar` should point into the `buildtools` folder. In every new shell, run both `source` lines, buildtools first, before using `bitbake`. Then run the build command from step F again.

G. Copy the images to Windows. The TFTP server and Tera Term run on Windows, so copy the two files you need out of the Ubuntu build folder into the `sample` folder of `mpfs-nmc-linux-bsp`. Replace `<path-to>` with the Windows folder that contains `mpfs-nmc-linux-bsp`.

```
cd ~/nmc-yocto/build/tmp-glibc/deploy/images/mpfs-icicle-kit
cp payload.bin /mnt/c/<path-to>/mpfs-nmc-linux-bsp/sample/payload-yocto.bin
cp mchp-base-image-mpfs-icicle-kit.rootfs.wic.gz /mnt/c/<path-to>/mpfs-nmc-linux-
bsp/sample/yocto.wic.gz
```

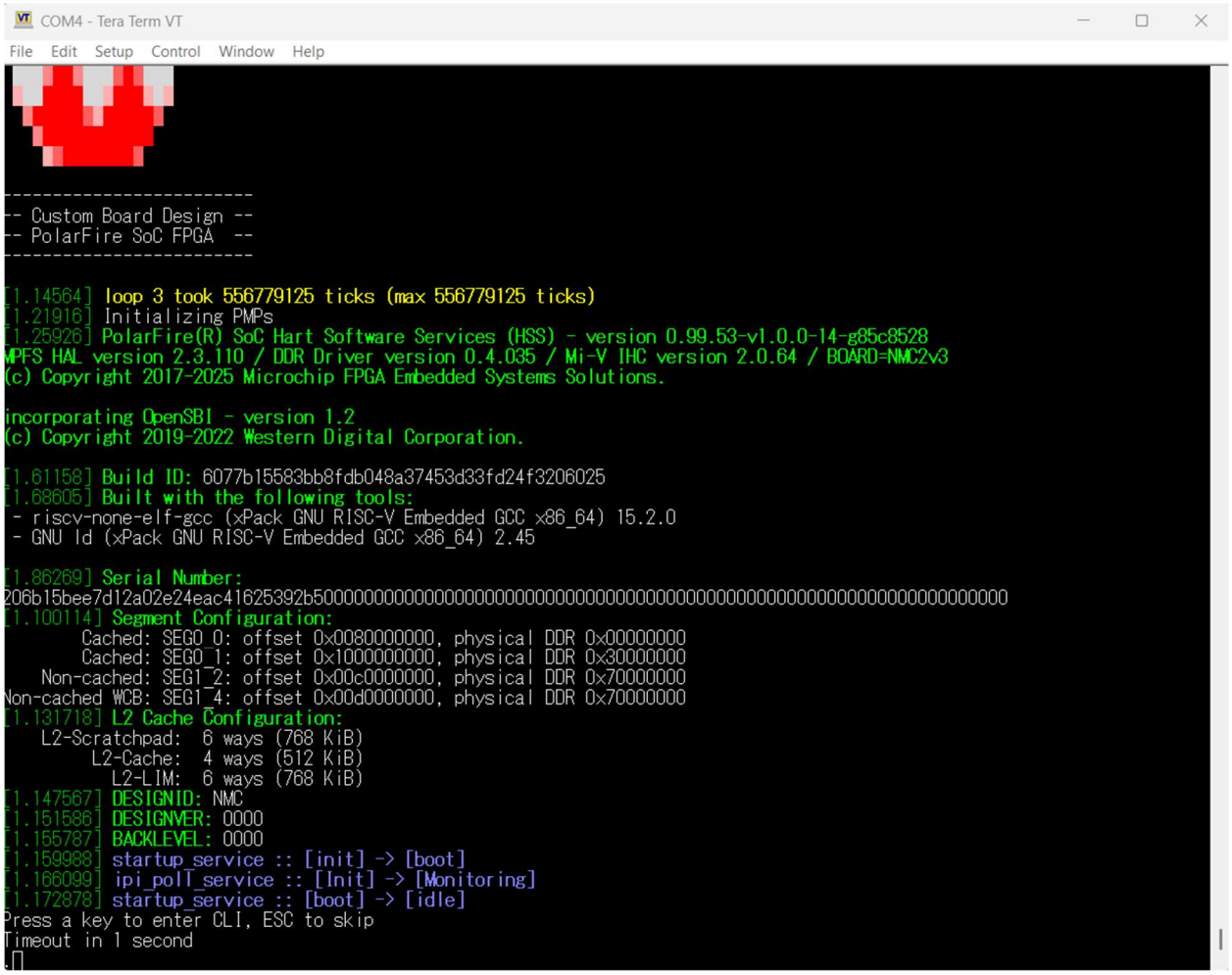
`payload-yocto.bin` goes to Step 6 (YMODEM). `yocto.wic.gz` is the compressed disk image you serve over TFTP in Step 7; point tftpd64 at the `sample` folder as in Step 7, and use `yocto.wic.gz` as the filename in the `tftpboot` command. Send the compressed file, because the uncompressed `.wic` (about 715 MB) is too large to load into U-Boot's RAM, and the compressed file (about 138 MB) transfers much faster. `explorer.exe .` opens the current Ubuntu folder in Windows Explorer if you prefer to copy the files by hand. The files keep the `-yocto` names so they do not overwrite the `payload.bin` that ships in that folder.

Step 6: Flash payload-yocto.bin to eMMC (HSS, YMODEM)

Interrupt the HSS boot in Tera Term on Window 1 (UART0), send `payload-yocto.bin` with YMODEM, and write it to eMMC. Use the file you copied to the `sample` folder in Step 5, part G.

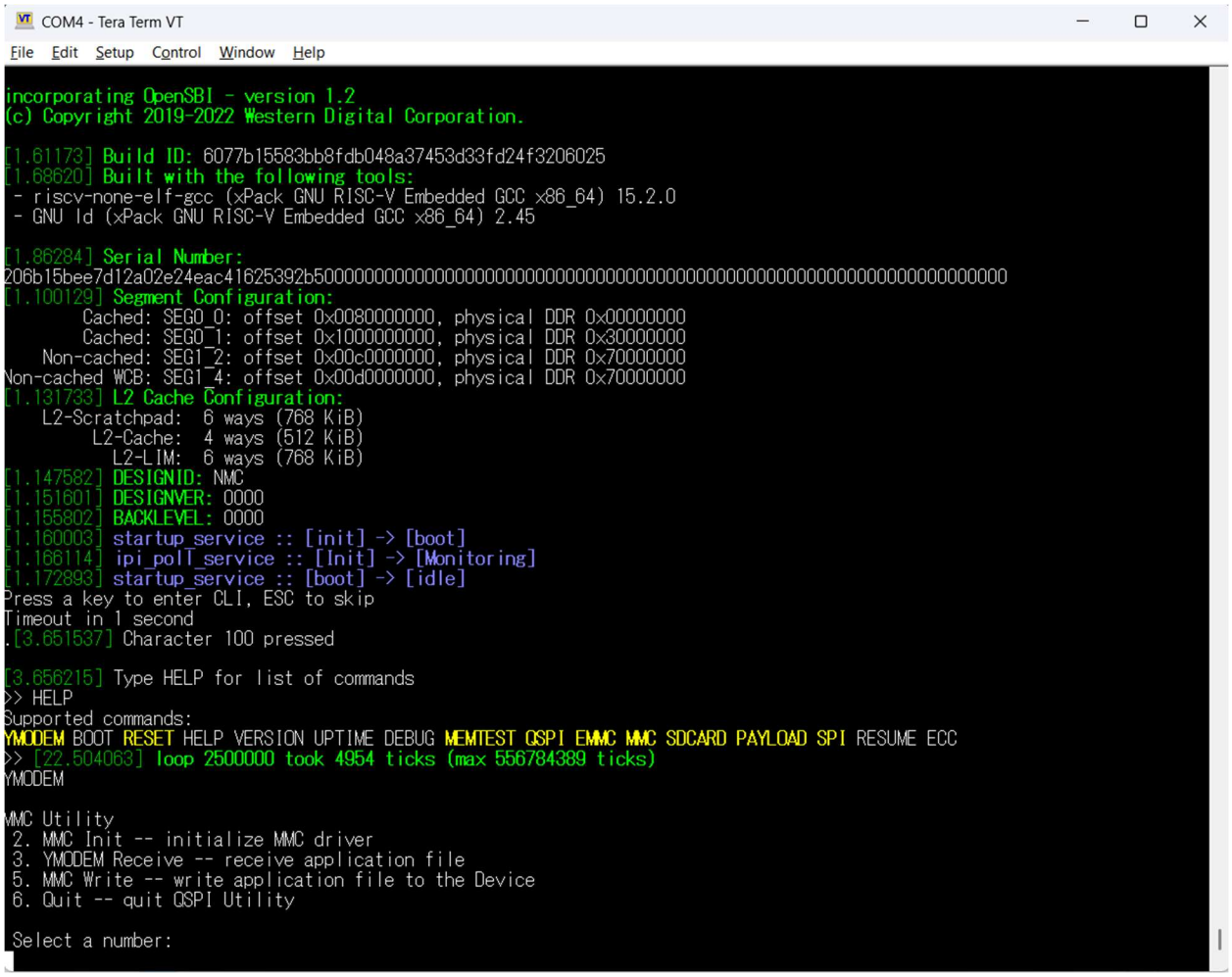
Get to the HSS prompt

1. Power cycle the board or press its Reset button.
2. Immediately press any key in Tera Term Window 1 (UART0) to stop auto-boot and reach the >> prompt.

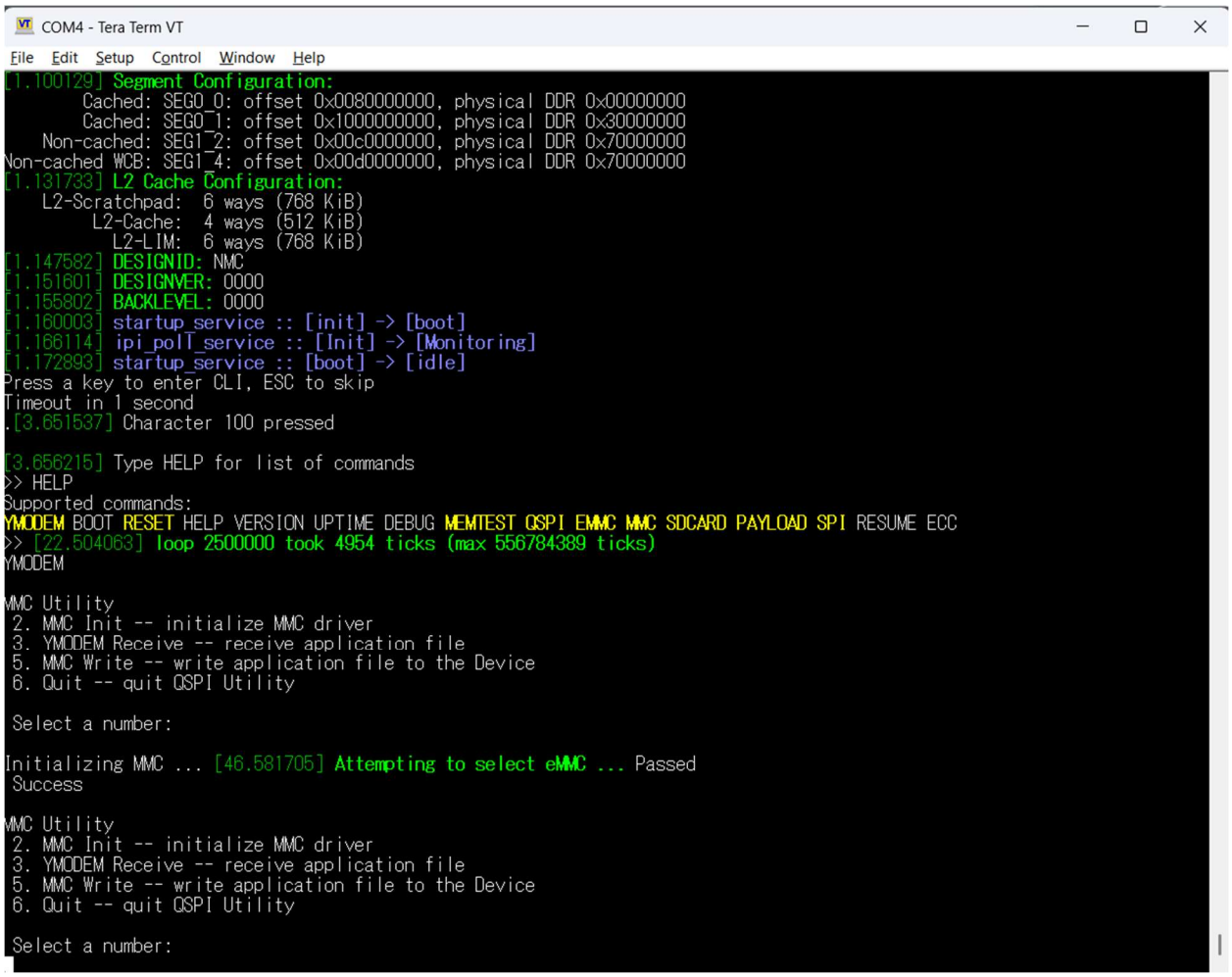


Receive the image

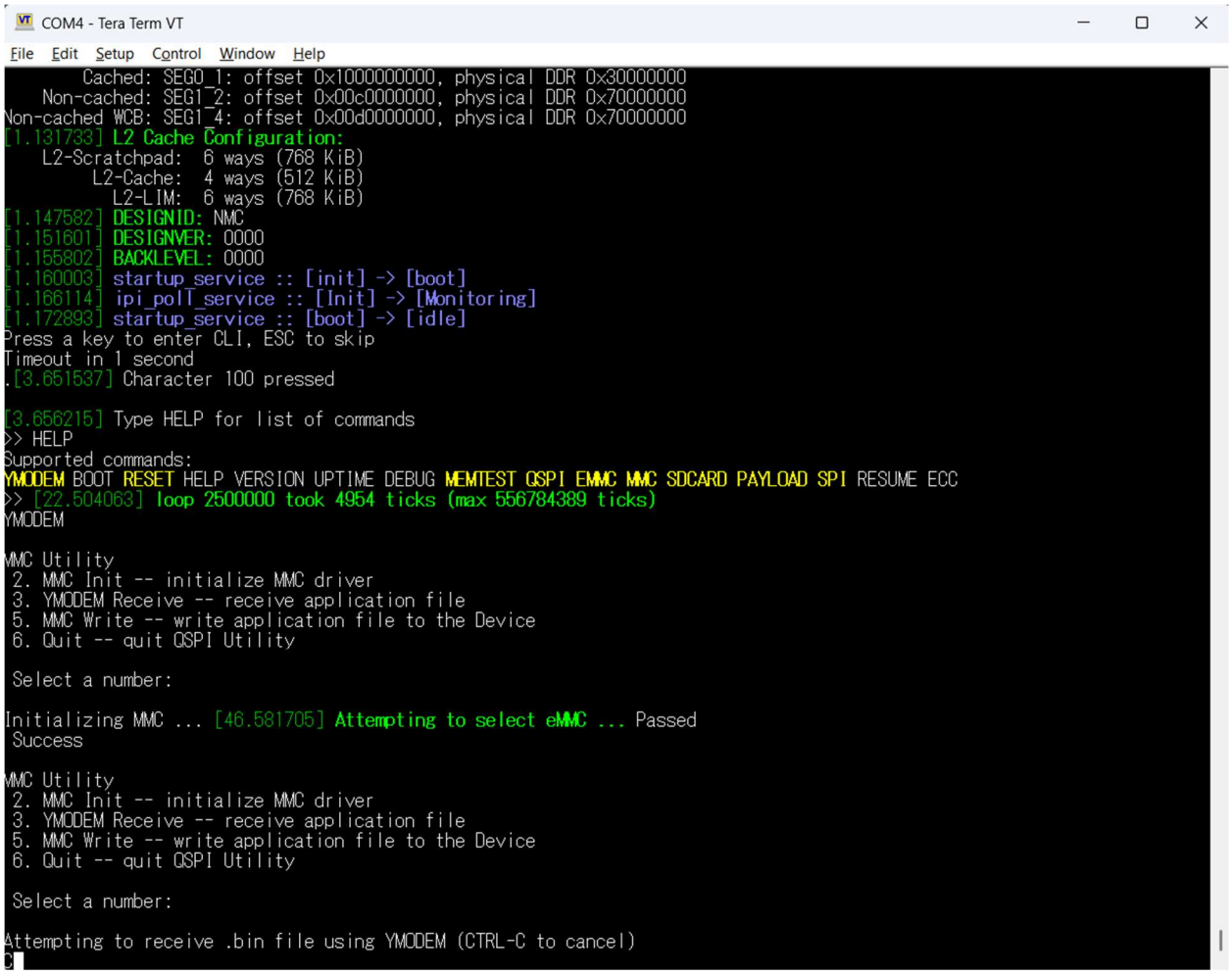
1. Type YMODEM and press Enter.



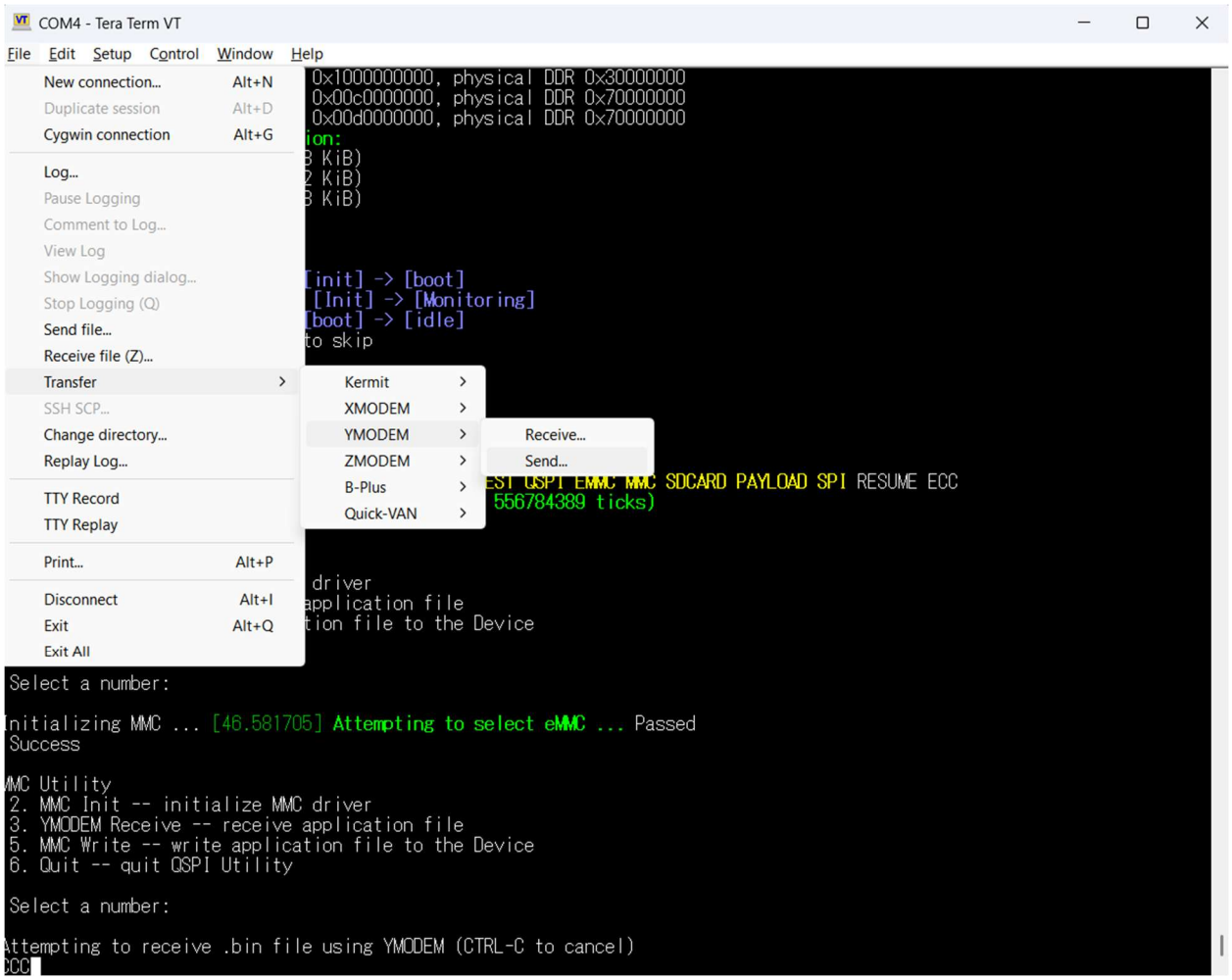
2. Type 2 and press Enter to run **MMC Init**.

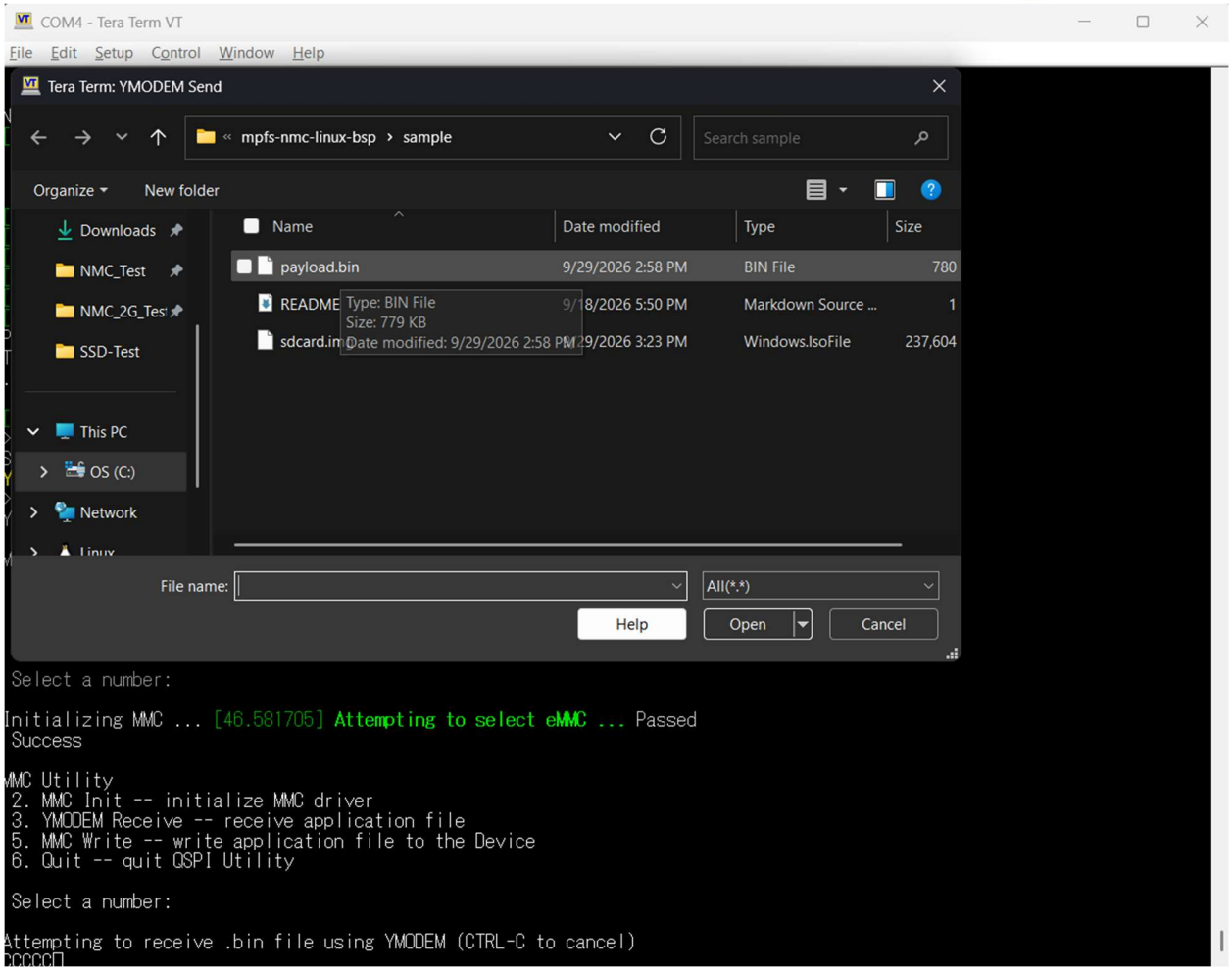


3. Type 3 and press Enter to select **YMODEM Receive**.

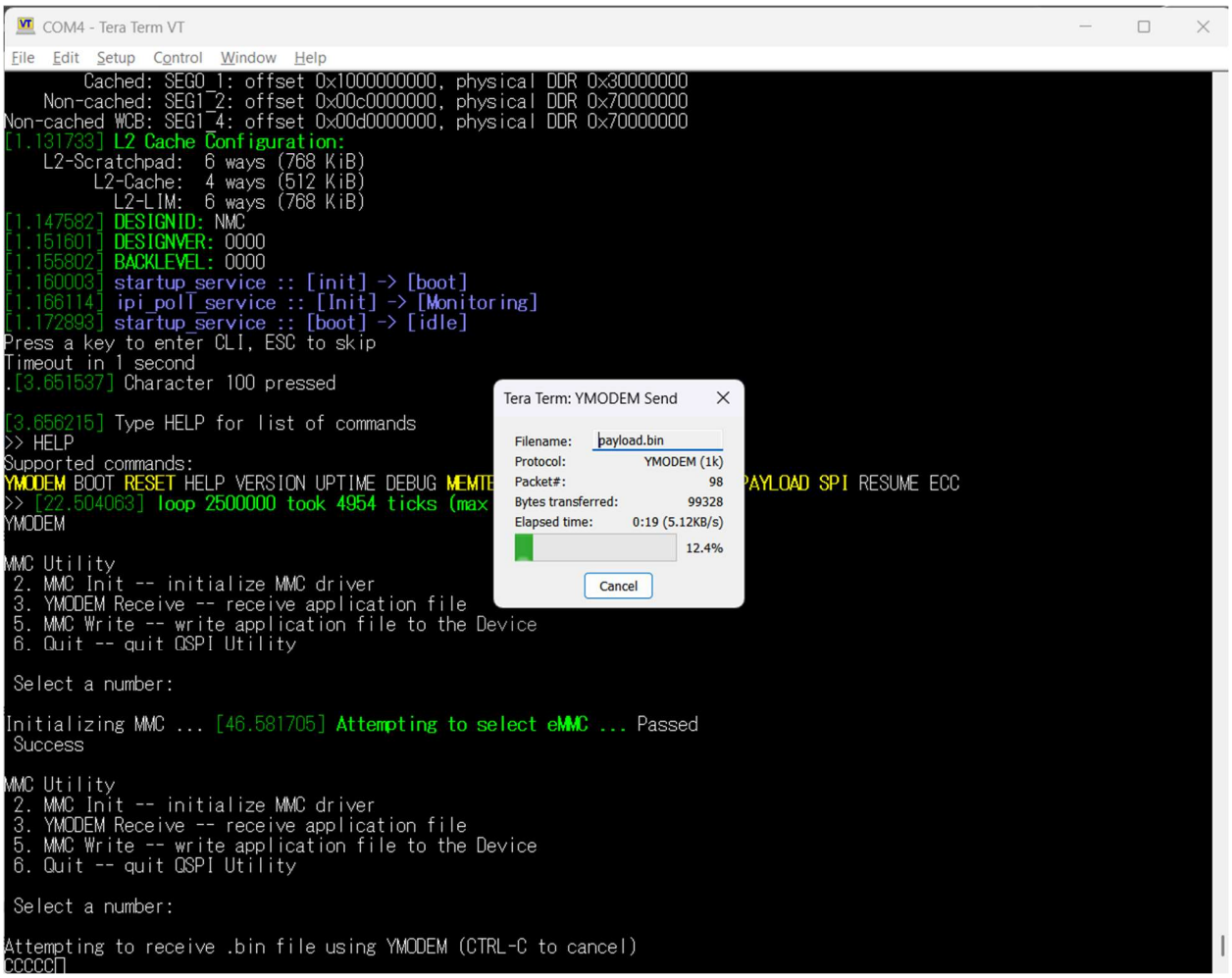


4. In Window 1 (UART0), choose **File → Transfer → YMODEM → Send...**, select `payload-yocto.bin`, and click **Open**.




COM4 - Tera Term VT
File Edit Setup Control Window Help
Tera Term: YMODEM Send
mpfs-nmc-linux-bsp › sample
Search sample
Organize
New folder
Downloads
NMC_Test
NMC_2G_Tes
SSD-Test
This PC
OS (C:)
Network
Name
Date modified
Type
Size
payload.bin
9/29/2026 2:58 PM
BIN File
780
README
Type: BIN File
Size: 779 KB
Date modified: 9/29/2026 2:58 PM
Markdown Source ...
1
sdcard.im
Windows.IsoFile
237,604
File name:
All(*.*)
Help
Open
Cancel
Select a number:
Initializing MMC ... [46.581705] Attempting to select eMMC ... Passed
Success
MMC Utility
2. MMC Init -- initialize MMC driver
3. YMODEM Receive -- receive application file
5. MMC Write -- write application file to the Device
6. Quit -- quit QSPI Utility
Select a number:
Attempting to receive .bin file using YMODEM (CTRL-C to cancel)

5. Wait for the transfer to finish.

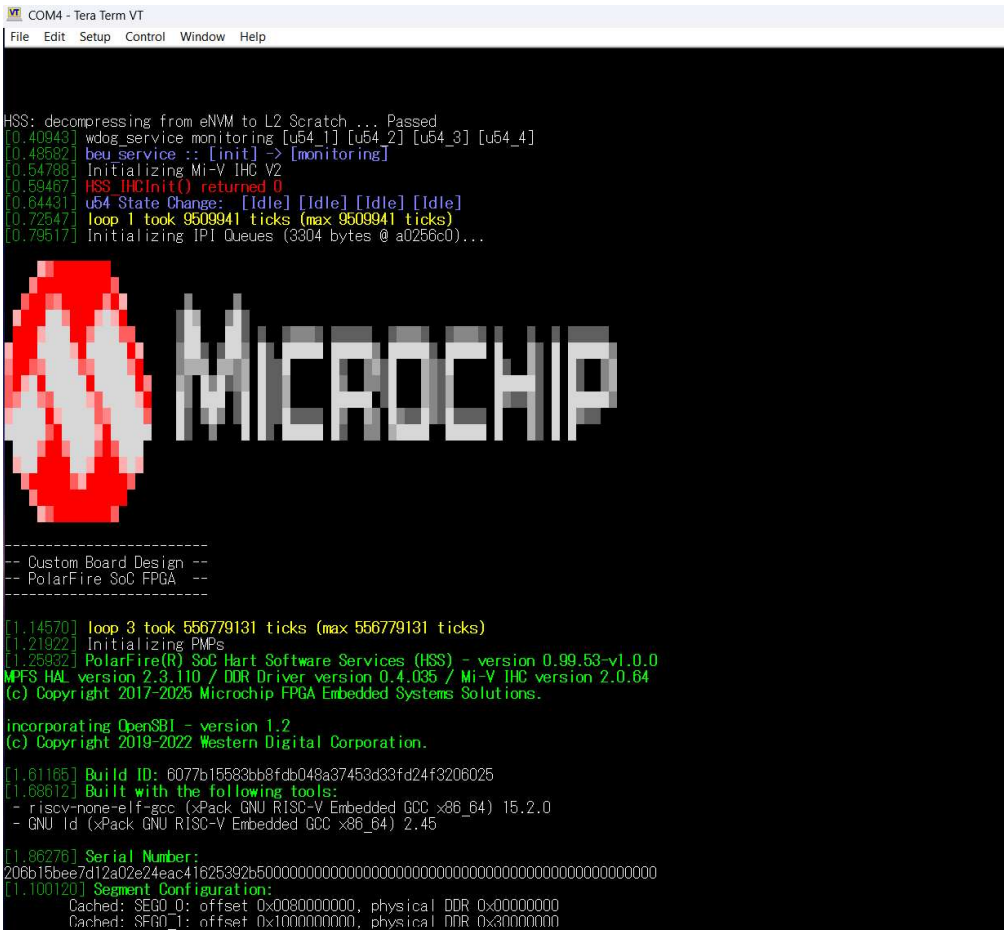


Write it permanently

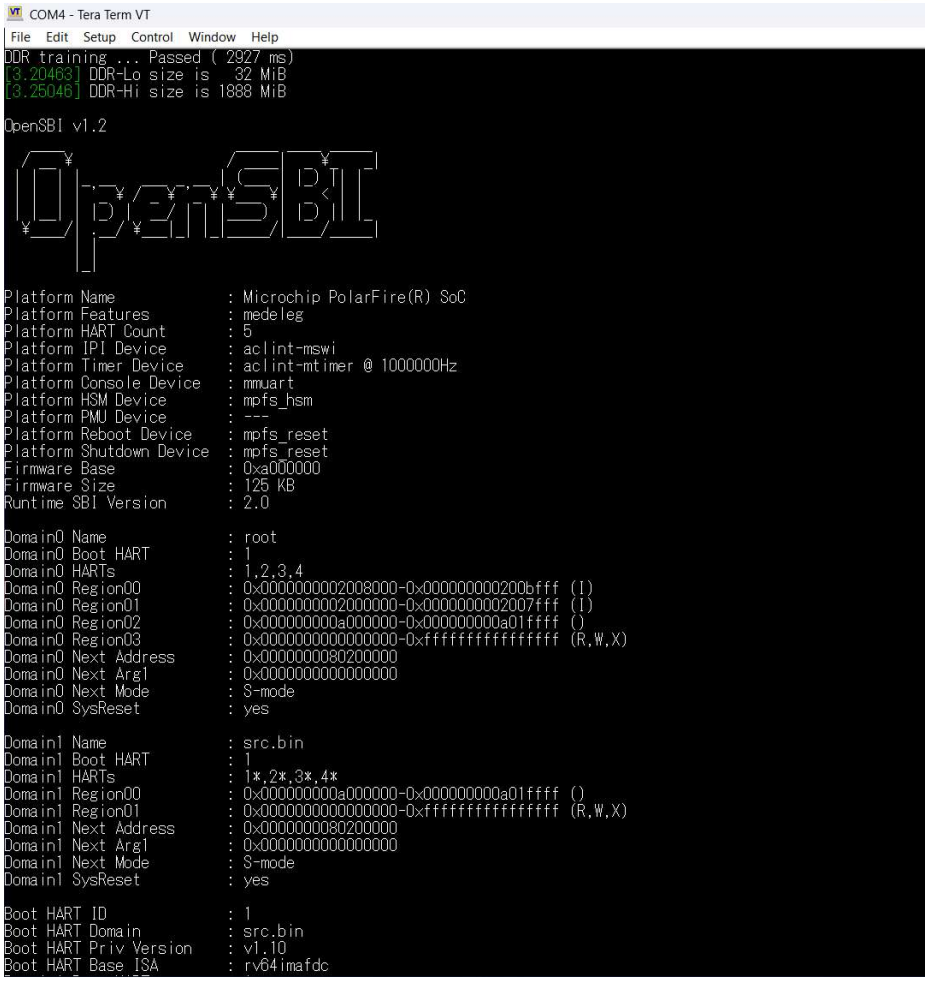
1. Type 5 and press Enter to run **MMC Write**.
2. Type 6 and press Enter to quit the utility.

Checkpoint: after a reset, you should land at a U-Boot prompt in Tera Term Window 2 (UART1). If not, resolve that before Step 7.

Both windows should look like this after `payload-yocto.bin` is written and the board resets, before you load the Linux image in Step 7.



UART0 (Window 1): the HSS boot log. It starts with HSS decompressing from eNVM, then shows the Microchip banner and the HSS version and build details.



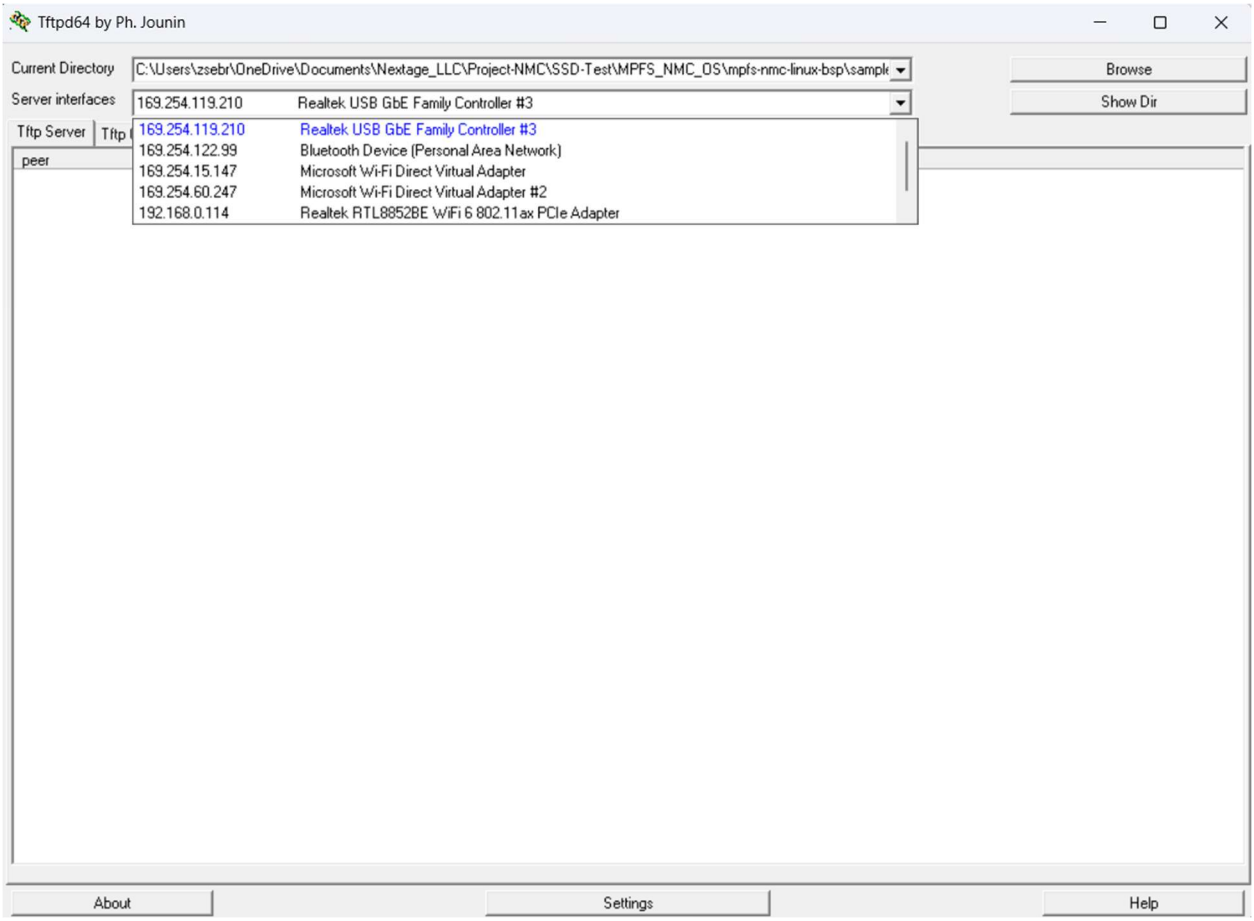
UART1 (Window 2): DDR training, then the OpenSBI banner with platform and domain details, as U-Boot starts.

Step 7: Flash yocto.wic.gz to eMMC (U-Boot, TFTP)

Serve `yocto.wic.gz` from the host over the Ethernet cable, then pull it into RAM from U-Boot and write it to eMMC with `gzwrite`. Use the file you copied to the `sample` folder in Step 5, part G.

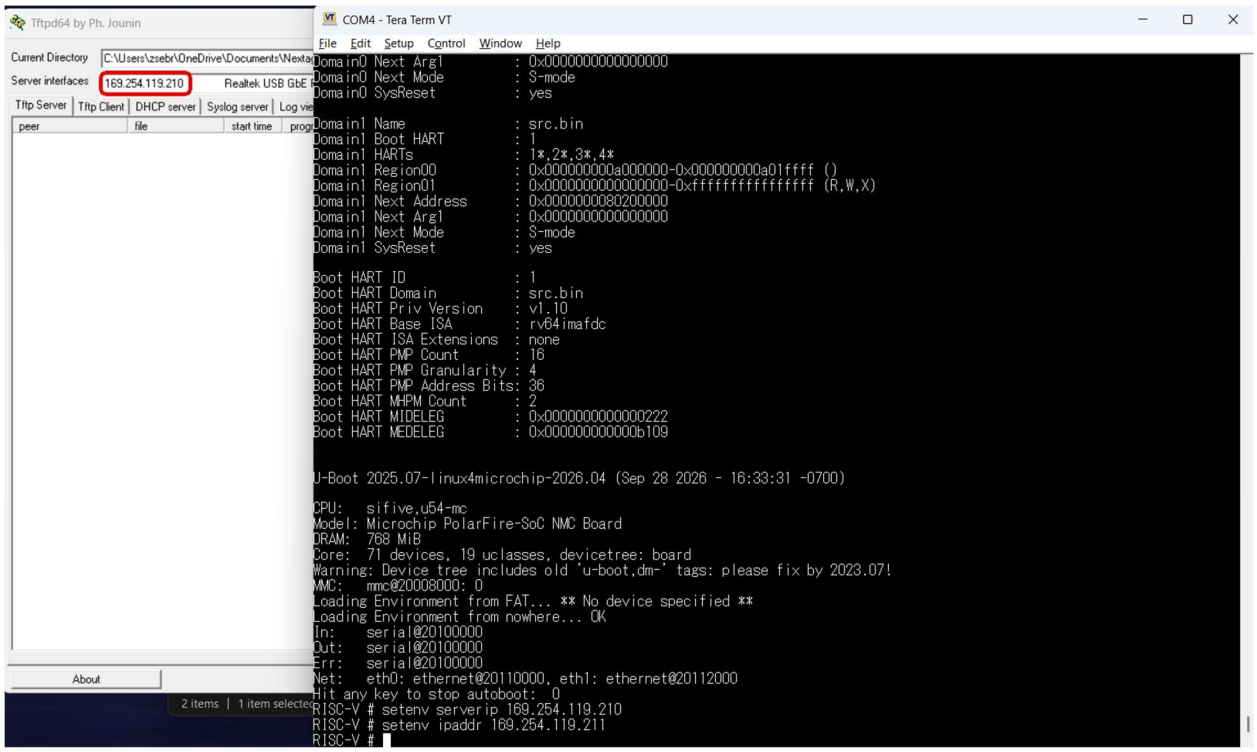
Set up the TFTP server on the host

1. Confirm the Ethernet cable runs from the board's J19 connector to the host PC. Use the eth0 end of the cable.
2. Start a TFTP server such as tftpd64. Point it at the folder containing `yocto.wic.gz` and select the host's Ethernet interface.
3. Use tftp64 to find your Ethernet controller on your machine and write down the IP address. You'll need it for the next steps. In this picture, the IP address of the ethernet port is 169.254.119.210. Yours will be different.



Set the network in Tera Term UART 1 (U-Boot)

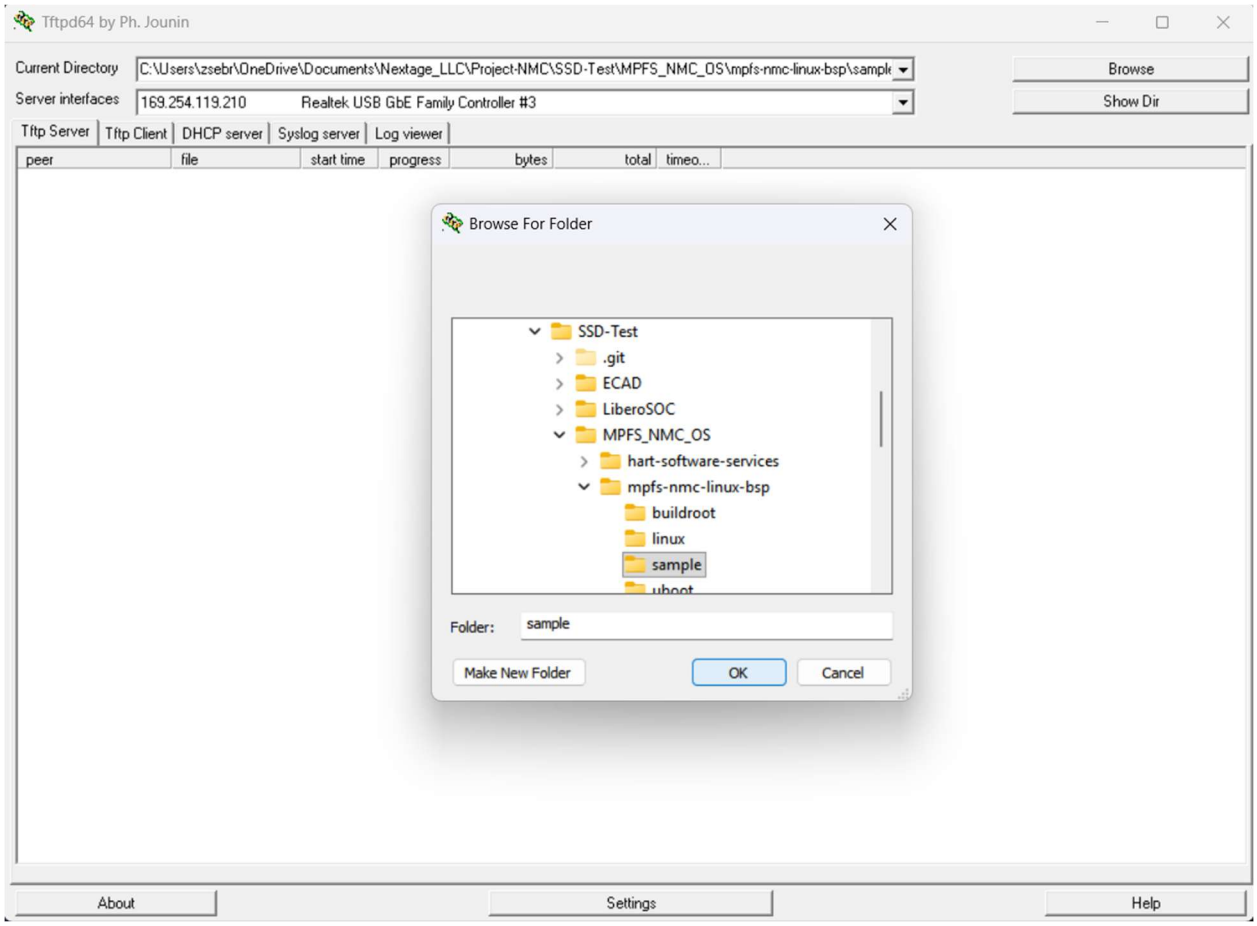
```
setenv serverip 169.254.69.98
setenv ipaddr 169.254.69.99
```

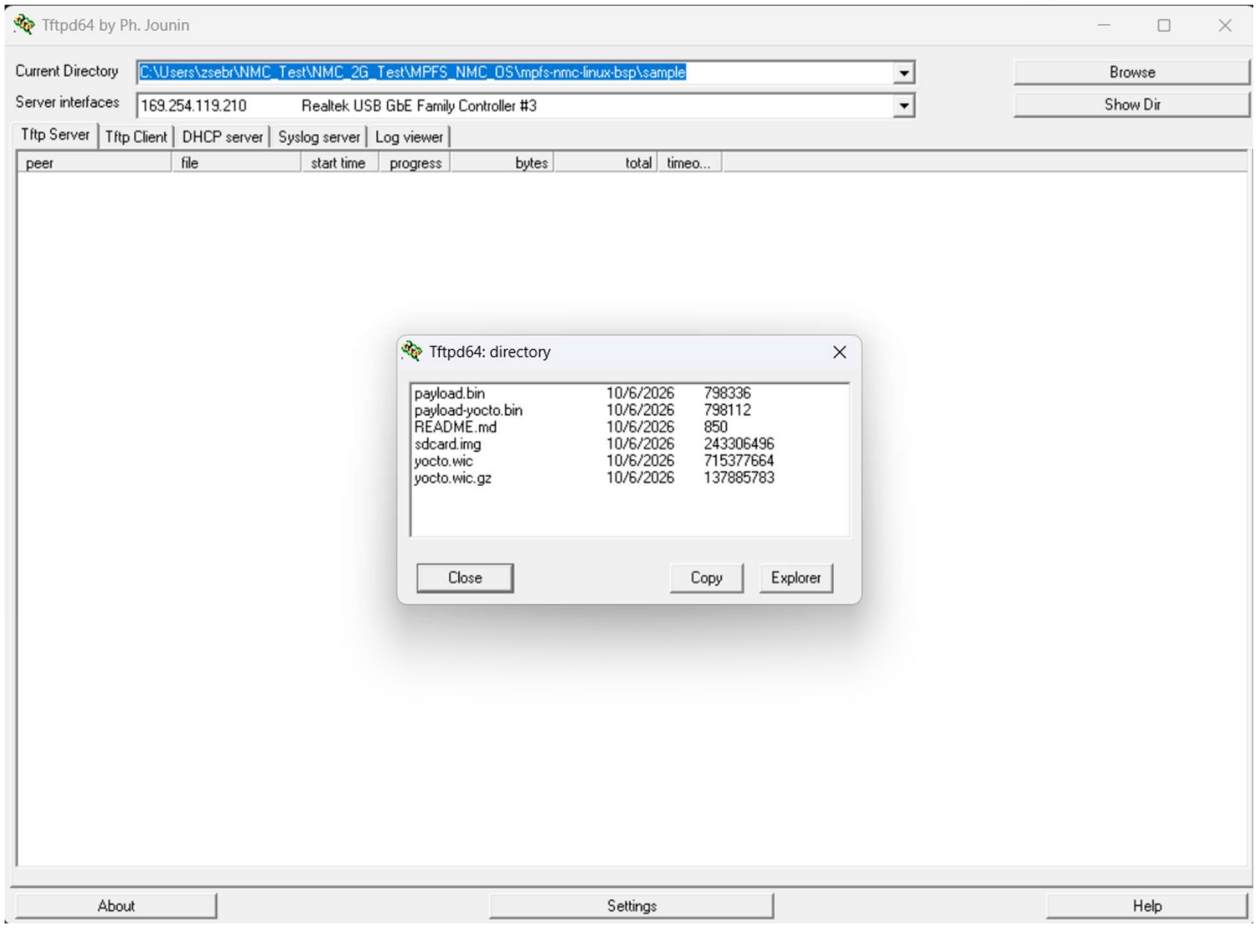


Set `serverip` to your host's Ethernet interface address, and `ipaddr` to an unused address on the same subnet. Set the host's Ethernet adapter to the `serverip` address manually.

Load and write the image

Copy yocto.wic.gz into your sample directory.





Write the image with gzwrite

The Yocto `.wic` image is about 715 MB, which does not fit in U-Boot's RAM at the load address, and an uncompressed `tftpboot` stops with `TFTP error: trying to overwrite reserved memory`. Send the compressed `yocto.wic.gz` (about 138 MB) instead, and write it with `gzwrite`, which decompresses while it writes. Use the same network settings as above, and check that the command exists first with `help gzwrite`.

```
tftpboot 0x84000000 yocto.wic.gz
mmc dev 0
gzwrite mmc 0 0x84000000 ${filesize}
```

`gzwrite` prints the number of bytes it wrote (715,377,664 in our build) and a CRC. Check the partition table with `part list mmc 0`, which should show three partitions named `boot`, `primary` and `root`. Then run `ls mmc 0:1`, which should list `boot.scr`, `fitImage` and `uboot.env`. Continue to Step 8.

Step 8: Boot Linux

Reset the board and let U-Boot boot Linux from the eMMC (Window 2, UART1), then log in.

Boot manually

```
fatload mmc 0:1 0x84000000 fitImage
bootm 0x84000000
```

The Yocto image boots Linux by itself after a reset: U-Boot runs the `boot.scr` from the boot partition, so there is no `bootcmd` to save. Use the commands above only if it stops at the U-Boot prompt.

Log in. Username `root`, no password.

Optional Step 9: Disable Ethernet to save power

After the image is flashed (Steps 6 to 8), the Ethernet PHYs keep drawing power even with no cable connected. Taking the Linux network interfaces down powers the PHYs down, which dramatically reduced the NMC's power draw in testing. This step is optional. TFTP reflashing in Step 7 is unaffected, because it runs in U-Boot with its own network stack.

A. Find the interface names. U-Boot calls the interfaces `eth0` and `eth1`, but Linux names them `end0` and `end1`. Run this at the Linux prompt on UART1:

```
ip -br link
```

B. Turn Ethernet off (until the next reboot).

```
ip link set end0 down
ip link set end1 down
ip -br link
```

Both interfaces should now show `DOWN`. Watch the supply current to confirm the drop.

C. Turn Ethernet back on.

```
ip link set end0 up
ip link set end1 up
```

Use the UART1 console to run this when Ethernet is off. Allow a few seconds for the link to come up. If you use a static IP address, add it again, or restart your DHCP client if you use DHCP.

D. Keep Ethernet off after a reboot. Create a small systemd service:

```
cat > /etc/systemd/system/eth-down.service << 'EOF'
[Unit]
Description=Bring Ethernet down to save power
After=network.target

[Service]
Type=oneshot
ExecStart=/sbin/ip link set end0 down
ExecStart=/sbin/ip link set end1 down

[Install]
WantedBy=multi-user.target
EOF
systemctl enable eth-down.service
```

Run `which ip` first and use that path in the two `ExecStart` lines if it is not `/sbin/ip`. Reboot, then run `ip -br link` to check that both interfaces come up `DOWN`. If something else in the image brings them back up, it must be told to leave `end0` and `end1` alone.

All steps, including step D, were tested on the NMC with the Yocto image.

Notes

1. With Ethernet off, UART1 is the way into the board.
2. Flashing a new image replaces the root filesystem, so the service in step D has to be created again after each reflash. To avoid that, add it to the Yocto layer so every image build includes it.
3. Reflashing over TFTP (Step 7) still works with Ethernet disabled in Linux.